

Data Preprocessing and Outlier Detection using Python

Data preprocessing is the process of preparing raw data so that it can be easily understood and correctly used for analysis or machine learning. Real-world data is often unorganized, incomplete, or not in a suitable format, which can lead to errors and incorrect results if used directly. Preprocessing involves cleaning the data, selecting useful information, removing unnecessary details, and organizing it into a structured and standard format. This step is very important because high-quality input data leads to accurate and meaningful results, while poorly prepared data can produce misleading conclusions. By reducing errors, improving reliability, and making data suitable for applying algorithms, data preprocessing acts as a strong foundation for effective data analysis and prediction.

For this preprocessing task, a **Hospital Stay dataset** was used (hpstay.csv). This dataset contains information related to patients' hospital stay, such as factors influencing the duration of stay in a hospital. The data is useful for understanding patterns related to hospital management and patient care. The main purpose of choosing this dataset was to prepare the data in a structured format so that it can later be used for analysis or prediction of hospital stay duration.

This report shows the steps included in preprocessing the hospital stay dataset in Google Colab.

Steps in Data Preprocessing

Step1: Import and Mount the Google Drive

```
 import os, sys
from google.colab import drive
drive.mount('/content/drive')
os.chdir("/content/drive/My Drive/Colab Notebooks")
sys.path.append("/content/drive/My Drive/Colab Notebooks")

... Mounted at /content/drive
```

This cell is used to connect Google Drive to Google Colab.

- google.colab is the environment where the notebook is running
- drive.mount() gives Colab permission to access files stored in Google Drive
- /content/drive is the location where Drive will appear inside Colab

Without this step:

- The notebook cannot access files saved in Drive
- The CSV file (hpstay.csv) would not be readable

Step 2: Import necessary libraries

```
import numpy as np  
import pandas as pd
```

This cell imports two important libraries:

- NumPy (np): Used for working with numbers and numerical data
- Pandas (pd): Used for reading, storing, and manipulating datasets in table form.

Step 3: Load the dataset

```
dataset=pd.read_csv('hpstay.csv')
```

- Reads hpstay.csv file from Drive.
- Loads it into a Pandas DataFrame called dataset.

Step 4: Display the Dataset

```
dataset
```

The cell displays the loaded dataset.

Step 5 : Selecting Independent and dependant Variables

```
▶ x=dataset.iloc[:,0:1].values  
y=dataset.iloc[:,1:2].values|
```

This cell selects the input data.

- `iloc` selects data based on position
- `0:1` means the first column
- `.values` converts it into numerical array format

This step chooses the factor that influences the result.

- `1:2` selects the second column
- This column contains the result or value to be analyzed.

The selected values are assigned to `x` and `y` variables indicating dependent and independent variables respectively.

Step 6 : Displaying the Dependant and Independent variables

▶ x|

This cell displays the selected input data.

▶ y

This cell displays the output values.

Purpose:

- To ensure correct output data is selected
- To confirm data is numerical and usable
- To verify correct column selection
- To visually inspect the values

Step 7: Handling Missing Values

```
from sklearn.impute import SimpleImputer  
imputer=SimpleImputer(missing_values=np.nan,strategy='mean')  
imputer1=imputer.fit_transform(x)
```

This step is used to handle missing values present in the dataset. The SimpleImputer tool from the scikit-learn library is imported to replace empty or missing values (np.nan) in the data. An imputer object is created with the strategy set to mean, which means that any missing value in the selected column will be replaced with the average value of that column. The fit_transform() method first calculates the mean from the existing data and then fills in the missing values accordingly. This process ensures that the dataset becomes complete, avoids errors during analysis, and makes the data suitable for further processing or machine learning tasks.

```
imputer1
```

This step displays the variable imputer1, which contains the updated version of the dataset after handling missing values. When this cell is executed, it shows the data where all previously missing entries have been replaced with appropriate values (in this case, the mean of the respective column). Displaying imputer1 helps verify that the imputation process was successful and confirms that the dataset is now complete, clean, and ready for further analysis or processing.

```
dataset.iloc[:,0:1]=imputer1
```

```
dataset
```

Assigning the updated values to the dataset and displaying the dataset.

Step 8: Handling Categorical Values

```
x1=dataset.iloc[:, :-1].values  
  
de=dataset.iloc[:, 1:2].values
```

This step is used to separate the dataset into input variables and output variables for further processing. The statement `x1 = dataset.iloc[:, :-1].values` selects all rows and all columns except the last one, and stores them as numerical values in `x1`. These columns represent the independent variables, which are the input features used for analysis or prediction. The next statement, `de = dataset.iloc[:, 1:2].values`, selects all rows from the second column only and stores them in `de`. This column represents the dependent variable, which is the outcome or result that depends on the input features. By performing this separation, the data becomes well-structured and ready for further analysis or machine learning tasks.

```
de
```

This step simply displays the variable `de`, which contains the selected dependent (output) data from the dataset. By running this cell, the values stored in `de` are shown on the screen, allowing us to verify that the correct column has been extracted. This helps confirm that the output variable has been properly separated from the input data and is in the correct numerical format, making it ready for further analysis or modeling.

```
from sklearn.preprocessing import OneHotEncoder  
ohe=OneHotEncoder()  
p=ohe.fit_transform(de).toarray()
```

```
p
```

This step is used to convert categorical (text-based) data into numerical form so that a computer can understand it. The OneHotEncoder is imported from the scikit-learn preprocessing library and an encoder object is created. The fit_transform(de) method first learns the different categories present in the variable de and then converts each category into a separate binary (0 or 1) column. The .toarray() method is used to display the encoded data in array format. Finally, displaying p shows the transformed data, where each category is represented numerically. This process is important because machine learning algorithms cannot work directly with text values, so one-hot encoding makes the data suitable for further analysis or modeling.

Step 9: Outlier Detection

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination='auto')
dataset['outlier'] = iso.fit_predict(dataset[['Hospital_Stay_Days']])

outliers = dataset[dataset['outlier'] == -1]
print(outliers)
```

This step is used to identify outliers in the dataset, specifically in the column representing hospital stay days. The IsolationForest algorithm is imported from the scikit-learn ensemble library and initialized with automatic contamination detection, which means the model automatically estimates how many unusual values may exist. The model is then applied to the Hospital_Stay_Days column using fit_predict(), which analyzes the data and labels each record as either normal or an outlier. These labels are stored in a new column called outlier, where a value of -1 indicates an outlier and 1 indicates a normal data point. In the next step, all records marked as outliers are filtered and stored in the variable outliers, and printing this variable displays only the unusual or abnormal hospital stay values. This process helps detect and analyze extreme or rare values that could affect the accuracy of further analysis or models.